

Ex. No.: 5A

Aim:

FIRST COME FIRST SERVE To implement First-come First-serve (FCFS) scheduling technique

Algorithm:

1. Get the number of processes from the user.
2. Read the process name and burst time.
3. Calculate the total process time.
4. Calculate the total waiting time and total turnaround time for each process
5. Display the process name & burst time for each process. 6. Display the total waiting time, average waiting time, turnaround time

Program:

```
#include <stdio.h>

int main()
{
    int n, i;
    int bt[10], wt[10], tat[10];
    int total_wt = 0, total_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for(i = 0; i < n; i++)
    {
        printf("p%d : ", i + 1);
        scanf("%d", &bt[i]);
    }

    "fcfs.c" 49L, 916B
```

```
Activities Terminal May 14 09:28
root@abhijith:~/abhi

}

wt[0] = 0;

for(i = 1; i < n; i++)
{
    wt[i] = wt[i - 1] + bt[i - 1];
}

for(i = 0; i < n; i++)
{
    tat[i] = wt[i] + bt[i];
    total_wt += wt[i];
    total_tat += tat[i];
}

printf("\nprocess\tburst time\tTurnaround Time\n");

for(i = 0; i < n; i++)
```

```
Activities Terminal May 14 09:29
root@abhijith:~/abhi

    total_wt += wt[i];
    total_tat += tat[i];
}

printf("\nprocess\tburst time\tTurnaround Time\n");

for(i = 0; i < n; i++)
{
    printf("p%d\t%d\t\t%d\t\t%d\n",
           i + 1, bt[i], wt[i], tat[i]);
}

printf("\nAverage waiting time = %.2f",
       (float)total_wt / n);

printf("\nAverage Turnaround time = %.2f\n",
       (float)total_tat / n);

return 0;
```

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Completed

First-Come,First-Served CPU Scheduling Algorithm

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- **Completion Time (CT):** The time at which a process finishes execution.
- **Turnaround Time (TAT):** Total time taken from arrival to completion.
- $TAT = CT - AT$
- **Waiting Time (WT):** The time a process spends waiting in the ready queue.
- $WT = TAT - BT$

Input Format

1. The first input line contains an integer representing the number of processes.

Select Language : C (GCC 9.2.0)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct { int id; int at; int bt; int ct; int tat; int wt; } Process;
5
6 int cmp_at(const void *a, const void *b) { Process *p1 = (Process*)a;
7 Process *p2 = (Process*)b; if (p1->at != p2->at) return p1->at - p2->at;
8 return p1->id - p2->id; // stable tie-breaker by original id
9 }
10
11
12 int main() { printf("Enter number of processes: \n"); int n; if
13 (scanf("%d", &n) != 1 || n <= 0) return 0;
14
15 Process *p = malloc(sizeof(Process) * n);
16 if (!p) return 0;
17
18 for (int i = 0; i < n; ++i) {
19     p[i].id = i + 1;
20     printf("Enter Arrival Time and Burst Time for P%d: \n", p[i].id);
21     if (scanf("%d %d", &p[i].at, &p[i].bt) != 2) { free(p); return 0; }
22     p[i].ct = p[i].tat = p[i].wt = 0;
23 }
24
```

Test Cases 2

Run Sample Cases

Run All Cases

Save

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Completed

First-Come,First-Served CPU Scheduling Algorithm

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- **Completion Time (CT):** The time at which a process finishes execution.
- **Turnaround Time (TAT):** Total time taken from arrival to completion.
- $TAT = CT - AT$
- **Waiting Time (WT):** The time a process spends waiting in the ready queue.
- $WT = TAT - BT$

Input Format

1. The first input line contains an integer representing the number of processes.

Select Language : C (GCC 9.2.0)

```
24
25 qsort(p, n, sizeof(Process), cmp_at);
26
27 int current_time = 0;
28 double total_tat = 0.0, total_wt = 0.0;
29
30 for (int i = 0; i < n; ++i) {
31     if (current_time < p[i].at) current_time = p[i].at;
32     current_time += p[i].bt;
33     p[i].ct = current_time;
34     p[i].tat = p[i].ct - p[i].at;
35     p[i].wt = p[i].tat - p[i].bt;
36     total_tat += p[i].tat;
37     total_wt += p[i].wt;
38 }
39
40 printf("\n");
41 printf("Average Turnaround Time: %.2f\n", total_tat / n);
42 printf("Average Waiting Time: %.2f\n", total_wt / n);
43
44 free(p);
45 return 0;
46
47
```

Test Cases 2

Run Sample Cases

Run All Cases

Save

Output:

```
[root@abhijith abhi]# vi fcfs.c
[root@abhijith abhi]# gcc fcfs.c -o fcfs
[root@abhijith abhi]# ./fcfs
Enter number of processes: 3
Enter burst time:
p1 : 5
p2 : 3
p3 : 2

process burst time      Turnaround Time
p1      5                0                5
p2      3                5                8
p3      2                8               10

Average waiting time = 4.33
Average Turnaround time = 7.67
[root@abhijith abhi]#
```

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

Test Cases 2

Run Sample Cases

Run All Cases

First-Come,First-Served CPU Scheduling Algorithm

Completed

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- **Completion Time (CT):** The time at which a process finishes execution.
- **Turnaround Time (TAT):** Total time taken from arrival to completion.
- $TAT = CT - AT$
- **Waiting Time (WT):** The time a process spends waiting in the ready queue.
- $WT = TAT - BT$

Input Format

1. The first input line contains an integer representing the number of processes.

Custom Test Case

> Custom Test

Sample Test Case

Test Case - 1

Input :

3
0 2
1 2
2 3

Expected Output :

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33
Average Waiting Time: 1.00

Output :

> Test Case - 2

Result:

Thus the program to implement FCFS scheduling technique in C has been successfully executed.

Ex. No.: 5b

Aim:

SHORTEST JOB FIRST To implement the Shortest Job First (SJF) scheduling technique

Algorithm:

1. Declare the structure and its elements.
2. Get number of processes as input from the user.
3. Read the process name, arrival time and burst time
4. Initialize waiting time, turnaround time & flag of read processes to zero.
5. Sort based on burst time of all processes in ascending order
6. Calculate the waiting time and turnaround time for each process.
7. Calculate the average waiting time and average turnaround time.
8. Display the results.

Program:

```
#include <stdio.h>

int main()
{
    int n, i, j;
    int bt[10], wt[10], tat[10];
    int temp;
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Burst time:\n");

    for(i = 0; i < n; i++)
    {
        printf("p%d : ", i + 1);
        scanf("%d", &bt[i]);
    }

    for(i = 0; i < n; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(bt[i] > bt[j])
            {
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

}
```

```

        bt[j] = temp;
    }
}

wt[0] = 0;

for(i = 1; i < n; i++)
{
    wt[i] = wt[i - 1] + bt[i - 1];
}

printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");

for(i = 0; i < n; i++)
{
    tat[i] = bt[i] + wt[i];

    avg_wt += wt[i];
    avg_tat += tat[i];

    printf("p%d\t%d\t%d\t%d\n",
           i + 1, bt[i], wt[i], tat[i]);
}

printf("\nAverage Waiting time = %.2f",
       avg_wt / n);

printf("\nAverage Turnaround Time = %.2f\n",
       avg_tat / n);

return 0;

```

Operating System - Coding | REC | 3 Feb

Course Progress
Go Back

Back To Practice

Completed

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulae:

- \$Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)\$
- \$Waiting Time (WT) = Turnaround Time (TAT) - Burst Time (BT)\$

Input Format

1. The first line of input contains an integer representing the number of processes

Q1

Select Language : C (GCC 9.2.0)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct {
5     int id;
6     int at; /* arrival time */
7     int bt; /* burst time */
8     int ct; /* completion time */
9     int tat; /* turnaround time */
10    int wt; /* waiting time */
11    int done;
12 } Process;
13
14 int main() {
15     int n;
16     printf("Enter number of processes: \n");
17     if (scanf("%d", &n) != 1 || n <= 0) return 0;
18
19     Process *p = malloc(n * sizeof(Process));
20     for (int i = 0; i < n; ++i) {
21         p[i].id = i + 1;
22         p[i].done = 0;
23         printf("Enter Arrival Time and Burst Time for P%d: \n", p[i].id);
24         if (scanf("%d %d", &p[i].at, &p[i].bt) != 2) { free(p); return 0; }

```

Test Cases
Run Sample Cases
Run All Cases

Operating System - Coding | REC | 3 Feb

Course Progress
Go Back

Back To Practice

Completed

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulae:

- Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)
- Waiting Time (WT) = Turnaround Time (TAT) - Burst Time (BT)

Input Format

1. The first line of input contains an integer representing the number of processes

Q1

Save

Select Language: C (GCC 9.2.0)

```

26 }
27
28 int completed = 0;
29 int time = 0;
30 double total_tat = 0.0, total_wt = 0.0;
31
32 /* If first arrival is later than 0, advance time to earliest arrival */
33 int earliest_at = p[0].at;
34 for (int i = 1; i < n; ++i) if (p[i].at < earliest_at) earliest_at = p[i].at;
35 if (earliest_at > time) time = earliest_at;
36
37 while (completed < n) {
38     int idx = -1;
39     int min_bt = 0x7fffffff;
40     /* find process with arrival <= time, not done, with smallest BT */
41     for (int i = 0; i < n; ++i) {
42         if (!p[i].done && p[i].at <= time) {
43             if (p[i].bt < min_bt || (p[i].bt == min_bt && p[i].at < p[idx].at)) {
44                 min_bt = p[i].bt;
45                 idx = i;
46             }
47         }
48     }
49     if (idx == -1) {

```

Test Cases 2

Run Sample Cases

Run All Cases

Operating System - Coding | REC | 3 Feb

Course Progress
Go Back

Back To Practice

Completed

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulae:

- Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)
- Waiting Time (WT) = Turnaround Time (TAT) - Burst Time (BT)

Input Format

1. The first line of input contains an integer representing the number of processes

Q1

Save

```

56 }
57
58 /* execute process idx */
59 time += p[idx].bt;
60 p[idx].ct = time;
61 p[idx].tat = p[idx].ct - p[idx].at;
62 p[idx].wt = p[idx].tat - p[idx].bt;
63 p[idx].done = 1;
64
65 total_tat += p[idx].tat;
66 total_wt += p[idx].wt;
67 completed++;
68 }
69
70 printf("\n");
71 printf("Average Turnaround Time = %.2f\n", total_tat / n);
72 printf("Average Waiting Time = %.2f\n", total_wt / n);
73
74 free(p);
75 return 0;
76 }

```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

Output:

```

[root@abhijith abhi]# vi fcfs.c
[root@abhijith abhi]# vi sjf.c
[root@abhijith abhi]# gcc sjf.c -o sjf
[root@abhijith abhi]# ./sjf
Enter number of processes: 3
Enter Burst time:
p1 : 6
p2 : 2
p3 : 4

Process Burst Time      Waiting Time      Turnaround Time
p1      2                0                2
p2      4                2                6
p3      6                6                12

Average Waiting time = 2.67
Average Turnaround Time = 6.67
[root@abhijith abhi]#

```

Operating System - Coding | REC | 3 Feb
Course Progress
Go Back

Back To Practice
Q1
Save

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulae:

- Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)
- Waiting Time (WT) = Turnaround Time (TAT) - Burst Time (BT)

Input Format

1. The first line of input contains an integer representing the number of processes

Custom Test Case

> Custom Test

Sample Test Case

Test Case - 1

Input :

```

4
0 6
0 8
0 7
0 3

```

Expected Output :

```

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:
Enter Arrival Time and Burst Time for P4:

Average Turnaround Time = 13.00
Average Waiting Time = 7.00

```

Output :

> Test Case - 2

Result:

Thus the program to implement the SJF scheduling technique has been successfully executed.

Ex. No.: 5c

Aim:

PRIORITY SCHEDULING To implement priority scheduling technique

Algorithm:

1. Get the number of processes from the user.
2. Read the process name, burst time and priority of process.
3. Sort based on burst time of all processes in ascending order based priority
4. Calculate the total waiting time and total turnaround time for each process
5. Display the process name & burst time for each process.
6. Display the total waiting time, average waiting time, turnaround time

Program:

```
#include <stdio.h>

int main()
{
    int n, i, j;
    int bt[10], pr[10], wt[10], tat[10];
    int temp;

    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Burst time and priority:\n");

    for(i = 0; i < n; i++)
    {
        printf("p%d Burst Time: ", i + 1);
        scanf("%d", &bt[i]);

        printf("p%d Priority: ", i + 1);
        scanf("%d", &pr[i]);
    }

    for(i = 0; i < n; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(pr[i] > pr[j])
            {
                temp = pr[i];
                pr[i] = pr[j];
                pr[j] = temp;
            }
        }
    }

    printf("ps.c" 67L, 1330B
```

```

        temp = bt[i];
        bt[i] = bt[j];
        bt[j] = temp;
    }
}

wt[0] = 0;

for(i = 1; i < n; i++)
{
    wt[i] = wt[i - 1] + bt[i - 1];
}

printf("\nProcess\tBurst time\tpriority\twaiting time\tTurnaround time\n");

for(i = 0; i < n; i++)
{
    tat[i] = bt[i] + wt[i];

    avg_wt += wt[i];
    avg_tat += tat[i];

    printf("p%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, bt[i], pr[i], wt[i], tat[i]);
}

printf("\nAverage waiting time = %.2f", avg_wt / n);

printf("\nAverage turnaround time = %.2f\n", avg_tat / n);

return 0;

```

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Completed

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

Select Language: C (GCC 9.2.0)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct { int pid; int at; int bt; int pr; int ct; int tat; int wt; int done; // flag
5 } Process;
6
7 int main() { int n; if (scanf("%d", &n) != 1 || n <= 0) return 0;
8
9 Process *p = (Process*)malloc(sizeof(Process) * n);
10 for (int i = 0; i < n; ++i) {
11     p[i].pid = i + 1;
12     if (scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pr) != 3) {
13         free(p);
14         return 0;
15     }
16     p[i].ct = p[i].tat = p[i].wt = 0;
17     p[i].done = 0;
18 }
19
20 // Print prompts exactly as required by judge (each followed by newline)
21 printf("Enter number of processes: \n");
22 for (int i = 0; i < n; ++i) {
23     printf("Enter AT, BT and Priority for P%d: \n", p[i].pid);
24 }

```

Test Cases 2

Run Sample Cases

Run All Cases

Operating System - Coding | REC | 3 Feb

Course Progress
Go Back

Back To Practice
Save

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

Select Language: C (GCC 9.2.0)

```

31 // if there are processes but earliest arrival > 0, advance time to earliest arrival when scheduling first
32 while (completed < n) {
33     // find all arrived and not done; pick one with highest priority (lowest pr)
34     int idx = -1;
35     for (int i = 0; i < n; ++i) {
36         if (!p[i].done && p[i].at <= curr_time) {
37             if (idx == -1) idx = i;
38             else {
39                 if (p[i].pr < p[idx].pr) idx = i;
40                 else if (p[i].pr == p[idx].pr) {
41                     // tie-breaker: earlier arrival time
42                     if (p[i].at < p[idx].at) idx = i;
43                     else if (p[i].at == p[idx].at) {
44                         // stable: lower pid (input order)
45                         if (p[i].pid < p[idx].pid) idx = i;
46                     }
47                 }
48             }
49         }
50     }
51     if (idx == -1) {
52         // no process has arrived yet; advance time to next arrival
53         int next_at = -1;
54         for (int i = 0; i < n; ++i) {

```

Test Cases 2
Run Sample Cases
Run All Cases

Operating System - Coding | REC | 3 Feb

Course Progress
Go Back

Back To Practice
Save

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

Select Language: C (GCC 9.2.0)

```

64 // schedule process idx (non-preemptive)
65 curr_time += p[idx].bt;
66 p[idx].ct = curr_time;
67 p[idx].tat = p[idx].ct - p[idx].at;
68 p[idx].wt = p[idx].tat - p[idx].bt;
69 p[idx].done = 1;
70
71 total_tat += p[idx].tat;
72 total_wt += p[idx].wt;
73 completed++;
74 }
75
76 double avg_tat = total_tat / n;
77 double avg_wt = total_wt / n;
78
79 // exact single blank line before averages
80 printf("\n");
81 printf("Average Turnaround Time = %.2f\n", avg_tat);
82 printf("Average Waiting Time = %.2f\n", avg_wt);
83
84 free(p);
85 return 0;
86
87 }

```

Test Cases 2
Run Sample Cases
Run All Cases

Output:

```

[root@abhijith abhi]# vi ps.c
[root@abhijith abhi]# gcc ps.c -o ps
[root@abhijith abhi]# ./ps
Enter number of processes: 3
Enter Burst time and priority:
p1 Burst Time: 5
p1 Priority: 2
p2 Burst Time: 3
p2 Priority: 1
p3 Burst Time: 4
p3 Priority: 3

Process Burst time    priority    waiting time    Turnaround time
p1      3             1           0              3
p2      5             2           3              8
p3      4             3           8             12

Average waiting time = 3.67
Average turnaround time = 7.67
[root@abhijith abhi]#

```

Operating System - Coding | REC | 3 Feb

Course Progress
Go Back

Back To Practice
Q1
Save

Completed

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

Test Cases 2

Custom Test Case

Custom Test

Sample Test Case

Test Case - 1

Input :

3
0 5 2
1 3 1
2 2 3

Expected Output :

Enter number of processes:
Enter AT, BT and Priority for P1:
Enter AT, BT and Priority for P2:
Enter AT, BT and Priority for P3:

Average Turnaround Time = 6.67
Average Waiting Time = 3.33

Output :

Test Case - 2

Run Sample Cases
Run All Cases

Result:

Thus the program to implement priority scheduling technique has been successfully executed.

Ex. No.: 5d

Aim:

ROUND ROBIN SCHEDULING To implement the Round Robin (RR) scheduling technique

Algorithm:

1. Declare the structure and its elements.
2. Get number of processes and Time quantum as input from the user.
3. Read the process name, arrival time and burst time
4. Create an array rem_bt[] to keep track of remaining burst time of processes which is initially copy of bt[] (burst times array)
5. Create another array wt[] to store waiting times of processes. Initialize this array as 0.
6. Initialize time : $t = 0$
7. Keep traversing the all processes while all processes are not done. Do following for i'th process if it is not done yet.
 - a- If $\text{rem_bt}[i] > \text{quantum}$ (i) $t = t + \text{quantum}$ (ii) $\text{bt_rem}[i] -= \text{quantum}$; b- Else // Last cycle for this process (i) $t = t + \text{bt_rem}[i]$; (ii) $\text{wt}[i] = t - \text{bt}[i]$ (iii) $\text{bt_rem}[i] = 0$;
8. Calculate the waiting time and turnaround time for each process.
9. Calculate the average waiting time and average turnaround time.
10. Display the results.

Program:

```
#include <stdio.h>

int main()
{
    int n, i, quantum;
    int bt[10], rem_bt[10], wt[10], tat[10];

    int t = 0;

    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("p%d : ", i + 1);

        scanf("%d", &bt[i]);

        rem_bt[i] = bt[i];

        wt[i] = 0;
    }

    printf("Enter time quantum: ");
    scanf("%d", &quantum);

    int done;

    do
    {
        done = 1;
        "rr.c" 79L, 1411B
```

```
        for(i = 0; i < n; i++)
        {
            if(rem_bt[i] > 0)
            {
                done = 0;

                if(rem_bt[i] > quantum)
                {
                    t += quantum;

                    rem_bt[i] -= quantum;
                }
                else
                {
                    t += rem_bt[i];

                    wt[i] = t - bt[i];

                    rem_bt[i] = 0;
                }
            }
        }

    } while(done == 0);

    printf("\nprocess\tburst time\tWaiting Time\tturnaround time\n");

    for(i = 0; i < n; i++)
    {
        tat[i] = bt[i] + wt[i];

        avg_wt += wt[i];
```

```

for(i = 0; i < n; i++)
{
    tat[i] = bt[i] + wt[i];

    avg_wt += wt[i];

    avg_tat += tat[i];

    printf("p%d\t%d\t%d\t%d\t%d\n",
        i + 1, bt[i], wt[i], tat[i]);
}

printf("\nAverage waiting time = %.2f", avg_wt / n);

printf("\nAverage turnaround time = %.2f\n", avg_tat / n);

return 0;
}

```

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Completed

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

Select Language: C (GCC 9.2.0)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct { int pid; int at; int bt; int pr; int ct; int tat; int wt; int done; // flag
5 } Process;
6
7 int main() { int n; if (scanf("%d", &n) != 1 || n <= 0) return 0;
8
9 Process *p = (Process*)malloc(sizeof(Process) * n);
10 for (int i = 0; i < n; ++i) {
11     p[i].pid = i + 1;
12     if (scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pr) != 3) {
13         free(p);
14         return 0;
15     }
16     p[i].ct = p[i].tat = p[i].wt = 0;
17     p[i].done = 0;
18 }
19
20 // Print prompts exactly as required by judge (each followed by newline)
21 printf("Enter number of processes: \n");
22 for (int i = 0; i < n; ++i) {
23     printf("Enter AT, BT and Priority for P%d: \n", p[i].pid);
24 }

```

Test Cases 2

Run Sample Cases

Run All Cases

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Completed

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

Select Language: C (GCC 9.2.0)

```

37 if (p[i].done == 0 && p[i].at <= curr_time) {
38     if (idx == -1) idx = i;
39     else {
40         if (p[i].pr < p[idx].pr) idx = i;
41         else if (p[i].pr == p[idx].pr) {
42             // tie-breaker: earlier arrival time
43             if (p[i].at < p[idx].at) idx = i;
44             else if (p[i].at == p[idx].at) {
45                 // stable: lower pid (input order)
46                 if (p[i].pid < p[idx].pid) idx = i;
47             }
48         }
49     }
50 }
51
52 if (idx == -1) {
53     // no process has arrived yet; advance time to next arrival
54     int next_at = -1;
55     for (int i = 0; i < n; ++i) {
56         if (!p[i].done) {
57             if (next_at == -1 || p[i].at < next_at) next_at = p[i].at;
58         }
59     }
60     if (next_at > curr_time) curr_time = next_at;

```

Test Cases 2

Run Sample Cases

Run All Cases

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

Completed

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

Select Language: C (GCC 9.2.0)

```
64 // schedule process idx (non-preemptive)
65 curr_time += p[idx].bt;
66 p[idx].ct = curr_time;
67 p[idx].tat = p[idx].ct - p[idx].at;
68 p[idx].wt = p[idx].tat - p[idx].bt;
69 p[idx].done = 1;
70
71 total_tat += p[idx].tat;
72 total_wt += p[idx].wt;
73 completed++;
74 }
75
76 double avg_tat = total_tat / n;
77 double avg_wt = total_wt / n;
78
79 // exact single blank line before averages
80 printf("\n");
81 printf("Average Turnaround Time = %.2f\n", avg_tat);
82 printf("Average Waiting Time = %.2f\n", avg_wt);
83
84 free(p);
85 return 0;
86
87 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Output:

```
[root@abhijith abhi]# vi rr.c
[root@abhijith abhi]# gcc rr.c -o rr
[root@abhijith abhi]# ./rr
Enter number of processes: 3
p1 : 10
p2 : 5
p3 : 8
Enter time quantum: 2

process burst time      Waiting Time      turnaround time
p1      10              13              23
p2      5               10              15
p3      8              13              21

Average waiting time = 12.00
Average turnaround time = 19.67
[root@abhijith abhi]#
```

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress [Go Back](#)

[Back To Practice](#)

Test Cases **2**

[Run Sample Cases](#)

[Run All Cases](#)

[Save](#)

Non-Preemptive Priority Scheduling algorithm

Completed

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

Custom Test Case

> Custom Test

Sample Test Case

Test Case - 1

Input :

3
0 5 2
1 3 1
2 2 3

Expected Output :

Enter number of processes:
Enter AT, BT and Priority for P1:
Enter AT, BT and Priority for P2:
Enter AT, BT and Priority for P3:

Average Turnaround Time = 6.67
Average Waiting Time = 3.33

Output :

> Test Case - 2

Result:

Thus the program to implement Round Robin scheduling technique has been successfully implemented.